Manual de despliegue de documentación

OpenKet

Sphinx + GitHub Pages

Última actualización: 23 de agosto de 2026

Índice

1 Cómo está organizado el repositorio 3
2 Requisitos previos 3
3 Preparar el entorno de trabajo 3
3.1 Clonar el repositorio 3
3.2 Crear un entorno virtual 3
3.3 Instalar las dependencias del paquete 4
3.4 Instalar las dependencias de la documentación 4
3.4.1 ¿Cómo se regenera `doc/requirements.txt` ? 4
4 Actualizar la documentación 5
4.1 Paso 1: Editar los archivos fuente 5
4.2 Paso 2: Compilar el HTML 5
4.3 Paso 3: Copiar el resultado a `docs/` 5
4.4 Paso 4: Revisar, hacer commit y push 5
4.5 Paso 5: Verificar el sitio publicado 6
5 Automatizar el flujo con un script 6
6 Resumen rápido (cheat-sheet) 6
7 Solución de problemas comunes 7

En este documento se explica cómo compilar y publicar la documentación **OpenKet**, alojada en `https://pbbmx.github.io/openket/`. Está pensado para cualquier persona que herede el mantenimiento del repositorio y no haya trabajado antes con Sphinx o con este flujo de publicación en particular.

No se asume experiencia previa con Sphinx: se explica desde la creación del entorno hasta el `push` final.

1 Cómo está organizado el repositorio

El repositorio `openket` combina el código del paquete de Python con su documentación. Los directorios relevantes para este manual son:

`doc/` contiene el **código fuente** de la documentación: los archivos `.rst`, `conf.py`, `Makefile`, `make.bat`, y las plantillas necesarias para que Sphinx genere el sitio.

`doc/_build/html` carpeta generada automáticamente al compilar. Es temporal: no se versiona a mano ni se edita, se regenera cada vez con `make html`.

`docs/` contiene el **HTML ya compilado**. Esta es la carpeta que GitHub Pages sirve públicamente. Nunca se edita a mano: su contenido es una copia de `doc/_build/html`, actualizada después de cada compilación.

Importante: este repositorio no usa una rama `gh-pages` separada. GitHub Pages está configurado para servir directamente la carpeta `/docs` de la rama `main` (verificable en **Settings → Pages** del repositorio en GitHub). Por eso el flujo consiste en compilar en `doc/`, dejar el resultado temporal en `doc/_build/html`, y copiar ese resultado a `docs/`, todo dentro de la misma rama.

2 Requisitos previos

- Tener Python 3 instalado (se recomienda 3.10 o superior).
- Tener `git` instalado, o usar GitHub Desktop.
- Tener acceso de escritura al repositorio `pbbmx/openket`.

El archivo `requirements.txt` en la raíz del repositorio contiene únicamente las dependencias necesarias para **ejecutar** el paquete `openket`, no las necesarias para **generar la documentación**. Sphinx y sus extensiones se instalan aparte, a partir de un segundo archivo: `doc/requirements.txt`, como se explica a continuación.

3 Preparar el entorno de trabajo

Estos pasos solo son necesarios la primera vez que se configura el proyecto en una máquina nueva.

3.1 Clonar el repositorio

```
git clone https://github.com/pbbmx/openket.git
cd openket
```

3.2 Crear un entorno virtual

Es buena práctica no instalar las dependencias directamente en el Python del sistema.

```
python3 -m venv venv
```

Activar el entorno:

```
# En Linux / macOS
source venv/bin/activate

# En Windows (PowerShell)
venv\Scripts\Activate.ps1
```

Cuando el entorno esté activo, la terminal debería mostrar el prefijo `(venv)` antes del prompt.

3.3 Instalar las dependencias del paquete

```
pip install -r requirements.txt
```

3.4 Instalar las dependencias de la documentación

Existe un archivo **aparte**, `doc/requirements.txt`, con únicamente lo necesario para compilar la documentación (Sphinx, el tema, y cualquier extensión que use `doc/conf.py`). Al estar ya generado con versiones fijas, solo hace falta instalarlo:

```
pip install -r doc/requirements.txt
```

3.4.1 ¿Cómo se regenera `doc/requirements.txt`?

Este archivo se crea con `pip freeze` desde un entorno donde **solo** estén instaladas las dependencias de documentación (para no arrastrar las del paquete `openket`, como numpy, scipy, etc.).

Si en el futuro se agrega una nueva extensión de Sphinx a `doc/conf.py`, hay que actualizarlo así:

```
#con el entorno virtual de documentación activado
pip install <paquete-nuevo>

#confirma que la documentación sigue compilando
cd doc
make html
cd ..

#regenera el archivo con las versiones exactas instaladas
pip freeze > doc/requirements.txt
```

Si `pip freeze` se ejecuta en un entorno que **también** tiene instalado `openket` y sus dependencias (numpy, scipy, etc.), esas quedarán mezcladas en `doc/requirements.txt`. Para mantenerlo limpio es importante usar un entorno virtual dedicado solo a documentación. Si no es práctico tener dos entornos, al menos revisa el archivo generado y elimina a mano las líneas que correspondan a dependencias del paquete y no de Sphinx.

4 Actualizar la documentación

Este es el proceso que se repite **cada vez** que se modifica algún archivo `.rst` (por ejemplo, al agregar una nueva función, actualizar una guía, corregir algún error, etc.).

4.1 Paso 1: Editar los archivos fuente

Los archivos `.rst` viven dentro de `doc/` (por ejemplo `doc/apidoc/functions.rst`, `doc/guide/guide-evolution.rst`). Edítalos con cualquier editor de texto.

4.2 Paso 2: Compilar el HTML

Con el entorno virtual activado, posiciónate en la carpeta donde vive `conf.py` y el `Makefile` y ejecuta:

```
make html
```

Esto genera el sitio compilado dentro de `doc/_build/html`. Sphinx recorrerá todos los `.rst`, ejecutará las directivas `autofunction` / `autoclass` sobre el código de `openket`, y producirá el HTML final ahí.

Si estás en Windows y no tienes `make` disponible, usa el script equivalente:

```
make.bat html
```

4.3 Paso 3: Copiar el resultado a `docs/`

Desde la raíz del repositorio:

```
cd ..
rm -rf docs/* #elimina el contenido de la versión anterior
cp -r doc/_build/html/. docs/ #copia de doc/ a docs/ todos los archivos de la
nueva versión
```

El `.` al final de la ruta de origen es importante: asegura que se copien también los archivos y carpetas que empiezan con guion bajo (`_static`, `_sources`, etc.), que de otra forma algunos comandos de copiado podrían omitir.

4.4 Paso 4: Revisar, hacer commit y push

Abre GitHub Desktop (o usa `git status` / `git add` / `git commit` desde terminal). Deberías ver cambios tanto en los archivos `.rst` de `doc/` como en todos los archivos `.html`, `.js` e `.inv` dentro de `docs/`. Esto es normal: ambos se versionan juntos.

```
git add doc docs
git commit -m "update documentación"
git push origin main
```

Con GitHub Desktop, el equivalente es: pestaña **Changes** → escribir el mensaje de commit → **Commit to main** → **Push origin**.

4.5 Paso 5: Verificar el sitio publicado

GitHub Pages puede tardar uno o dos minutos en reflejar el cambio. Visita:

```
https://pbbmx.github.io/openket/
```

y confirma que la página muestre el contenido actualizado. Si no se actualiza después de unos minutos, revisa el estado de **Settings → Pages**.

5 Automatizar el flujo con un script

Para evitar escribir los mismos comandos cada vez, si usas Linux se recomienda crear un script `deploy.sh` en la raíz del repositorio:

```
#!/bin/bash
set -e

echo "compilando documentación..."
cd doc
make html
cd ..

echo "copiando HTML a docs/..."
rm -rf docs/*
cp -r doc/_build/html/. docs/

echo "Listo. Revisa los cambios y haz commit/push."
```

Dar permisos de ejecución una sola vez:

```
chmod +x deploy.sh
```

Incluir al `.gitignore` para evitar archivos innecesarios en el repositorio, agregando la línea:

```
deploy.sh
```

Uso posterior, cada vez que se editan los `.rst` :

```
./deploy.sh
```

6 Resumen rápido (cheat-sheet)

1. Activar el entorno: `source venv/bin/activate`
2. Editar los `.rst` dentro de `doc/`
3. Compilar: `cd doc && make html`
4. Copiar: `cd .. && rm -rf docs/* && cp -r doc/_build/html/. docs/`

5. Commit y push (GitHub Desktop o `git add . && git commit -m "..." && git push`)
6. Verificar en `https://pbbmx.github.io/openket/`

7 Solución de problemas comunes

`make: command not found` instala `make` (en Windows usa `make.bat html` en su lugar, o instala `make` vía Chocolatey/WSL).

Errores de importación al ejecutar `make html` falta instalar `doc/requirements.txt`, o quedó desactualizado tras agregar una extensión nueva a `conf.py`. Reinstala con `pip install -r doc/requirements.txt` y, si el paquete faltante no está ahí, agrégalo y regenera el archivo (ver sección 5.5).

El sitio en GitHub Pages no refleja los cambios confirma que el `push` incluyó la carpeta `docs/` actualizada (no solo los `.rst`), y espera un par de minutos a que GitHub Pages reconstruya.

`autofunction` / `autoclass` no encuentra el módulo asegúrate de que el entorno virtual tiene instalado el propio paquete `openket` (por ejemplo con `pip install -e .` desde la raíz del repositorio) para que Sphinx pueda importarlo al generar la documentación de la API.